

BANCO DE PREGUNTAS

# Preguntas y respuestas

*Con el reparto de quién contesta cada tipo*



Abierto durante la transmisión  
Demostrador y expositor

# Contenido

|                                |   |
|--------------------------------|---|
| Las cinco que siempre salen    | 3 |
| Mecánica y sintaxis · D        | 4 |
| Uso y operación · D            | 5 |
| Adopción y equipo · E          | 5 |
| Comparaciones · E              | 6 |
| Preguntas incómodas            | 6 |
| Si nadie pregunta nada         | 7 |
| Lo que se anota para el correo | 7 |

Las preguntas que salen en un webinar de Skills, con la respuesta ya redactada y con quién la contesta.

La columna de la izquierda de cada tabla dice **D** cuando la contesta el demostrador y **E** cuando la contesta el expositor. El criterio es simple:

- **D • demostrador** — mecánica: rutas, sintaxis, permisos, por qué no se dispara, qué hace tal comando.
- **E • expositor** — adopción, equipo, gobierno, costo, comparaciones con otras herramientas, y todo lo que empiece con "en mi empresa".

Cuando reparten en voz alta, dilo: "esa es de mecánica, <nombre>". La audiencia entiende el reparto de inmediato y deja de dirigir todo a una sola voz.

## Las cinco que siempre salen

Estas cinco aparecen en prácticamente todo webinar de este tema. Vale la pena tenerlas casi memorizadas.

### 1 • ¿En qué se diferencia de un CLAUDE.md? • E

En el costo, y es toda la diferencia. Lo que está en `CLAUDE.md` se carga en **todas** sus conversaciones, la use o no. El cuerpo de una skill solo se carga cuando se usa. La pregunta que decide entre uno y otro: ¿hace falta en cada conversación, o solo en algunas? Si es en algunas, es skill.

No digas que las skills reemplazan a `CLAUDE.md`. Conviven, y la mayoría de los proyectos maduros tienen los dos.

### 2 • ¿Cuántas skills puedo tener antes de que sea un problema? • D

El costo permanente es la suma de las descripciones, no de los cuerpos. Con cincuenta skills tienen cincuenta descripciones de dos líneas en contexto, y eso es manejable. El problema real no es el número: es el **traslape**. Dos skills con descripciones parecidas compiten, y ahí sí empiezan a disparar la equivocada.

Y el consejo práctico:

Si tienen más de quince, córranle `/context` y vean lo que están ocupando de verdad antes de preocuparse.

### 3 • ¿Se pueden compartir con el equipo? • E

Sí, y es lo que más cambia las cosas. Una skill en `.claude/skills/` del repositorio se versiona con el código: quien clona hereda su criterio sin que nadie le explique nada. No hay instalación ni aviso. Y como es un archivo de texto, se revisa en un PR como cualquier otro cambio. Eso significa que el criterio del equipo deja de vivir en la cabeza de dos personas.

### 4 • ¿Por qué no se dispara la mía? • D

Es la pregunta más frecuente del webinar y casi siempre es lo mismo:

Noventa por ciento de las veces, la descripción. Tres cosas en orden: ¿está en tercera persona? ¿dice cuándo usarla, no solo qué hace? ¿usa las palabras que ustedes teclean de verdad, o las del manual? Si las tres están bien y sigue sin disparar, córranle `/doctor` : puede ser frontmatter roto, y ahí la skill carga sin metadatos.

## 5 • ¿Esto funciona con cualquier lenguaje o stack? • D

Sí, porque una skill es texto. No hay nada específico de lenguaje en el mecanismo. Lo que van a ver hoy son ejemplos con git, markdown y shell justamente por eso: para que no dependa de su stack. Donde sí importa el stack es en el contenido que ustedes escriban adentro, y eso es exactamente lo que nadie más puede escribir por ustedes.

## Mecánica y sintaxis • D

| PREGUNTA                                                                  | RESPUESTA                                                                                                                                                                         |
|---------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>D</b> ¿El campo <code>name</code> es obligatorio?                      | No. En skills personales y de proyecto es solo la etiqueta del listado. <b>El comando sale del nombre del directorio.</b> Es el error más extendido del tema                      |
| <b>D</b> ¿Qué pasa si dos skills se llaman igual?                         | Gana la de mayor precedencia: empresarial sobre personal, personal sobre proyecto                                                                                                 |
| <b>D</b> ¿Cuánto puede medir la descripción?                              | 1,536 caracteres, <b>combinados</b> entre <code>description</code> y <code>when_to_use</code> . Si encuentran 1,024 escrito por ahí, está desactualizado                          |
| <b>D</b> ¿Qué tan largo puede ser el cuerpo?                              | Técnicamente mucho, pero recuerden que se queda en contexto tras invocarla. Por debajo de 100 líneas es una buena disciplina; si necesitan más, va a un <code>reference.md</code> |
| <b>D</b> ¿ <code>allowed-tools</code> limita lo que la skill puede hacer? | No. <b>Otorga</b> permiso previo, no restringe. Y se limpia en el mensaje siguiente                                                                                               |
| <b>D</b> ¿Puedo evitar que una skill se dispare sola?                     | Sí, con <code>disable-model-invocation: true</code> . Eso quita la descripción del contexto por completo y solo se puede invocar con diagonal                                     |
| <b>D</b> ¿Puedo anidar archivos de referencia?                            | Un solo nivel. Las cadenas de dos niveles funcionan a veces y fallan otras, que es peor que fallar siempre                                                                        |
| <b>D</b> ¿Las rutas son relativas a qué?                                  | Usen <code>\${CLAUDE_SKILL_DIR}</code> , siempre. Funciona en su máquina, en la de su compañero y en un contenedor de CI                                                          |
| <b>D</b> ¿Puedo meter un script en otro lenguaje?                         | Sí, cualquiera que su máquina pueda ejecutar. El script se ejecuta, no se lee, así que el lenguaje da igual                                                                       |
| <b>D</b> ¿Qué pasa si el comando del contexto dinámico falla?             | <b>Se aborta la invocación completa</b> de la skill. Por eso en producción se escribe <code>! git \ true``<br/>diff HEAD \</code>                                                 |

## Uso y operación · D

| PREGUNTA                                                       | RESPUESTA                                                                                                                      |
|----------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| D ¿Cómo sé si una skill se activó?                             | Aparece en la traza de la respuesta. Y si quieren verificarlo, <code>/context</code> les dice qué está cargado ahora mismo     |
| D ¿Puedo forzar una skill?                                     | Sí, con <code>/nombre-del-directorio</code> . Es lo que se hace para probar el cuerpo aislado de la descripción                |
| D ¿Puedo usar dos skills en la misma conversación?             | Sí, y se acumulan en contexto. Es otra razón para tener cuerpos cortos                                                         |
| D ¿Qué pasa con las skills cuando se compacta la conversación? | Se vuelve a adjuntar la invocación más reciente. Hay un presupuesto acotado, así que una skill muy larga puede quedar truncada |
| D ¿Tengo que reiniciar Claude Code al crear una skill?         | No para skills personales y de proyecto: se leen del disco. Si no aparece, es frontmatter                                      |
| D ¿Puedo probar una skill sin ensuciar mi repo?                | Sí, y es lo recomendable: sesión nueva en un directorio de prueba. Es el Claude B del ciclo                                    |

## Adopción y equipo · E

| PREGUNTA                                     | RESPUESTA                                                                                                                                              |
|----------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| E ¿Cuántas debería escribir para empezar?    | <b>Una.</b> Y usarla tres veces. Quien sale de un webinar y escribe cinco el mismo día, no usa ninguna                                                 |
| E ¿Cómo convenzo a mi equipo?                | No los convenzas: mete una al repositorio y deja que la hereden sin avisar. La conversación es mucho más fácil después de que ya funcionó              |
| E ¿Quién debería escribirlas?                | Quien tenga el criterio, no quien tenga el tiempo. Una skill es criterio escrito, y si la escribe alguien que no lo tiene, queda vacía                 |
| E ¿Cómo evito que se vuelvan un basurero?    | Fecha de revisión y dueño. Una skill sin dueño a los seis meses es deuda. En el kit viene una skill de auditoría para esto                             |
| E ¿Se pueden revisar en PR?                  | Es un archivo de texto: se revisa exactamente igual que el código. Y es donde conviene discutir el criterio, no en una junta                           |
| E ¿Sirven para onboarding?                   | Es uno de los mejores usos. La persona nueva hereda el criterio del equipo sin que nadie se lo explique dos veces                                      |
| E ¿Esto no es documentación con otro nombre? | La documentación la lee un humano si se acuerda de que existe. Una skill se aplica sola en el momento en que hace falta. La diferencia es cuándo llega |

## Comparaciones · E

| PREGUNTA                            | RESPUESTA                                                                                                                                                                             |
|-------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| E ¿Skill o comando de diagonal?     | El comando lo invoca el usuario a propósito. La skill se dispara sola cuando aplica. Si quieren control explícito, comando; si quieren que funcione sin acordarse, skill              |
| E ¿Skill o subagente?               | La skill le da contexto al agente que ya está trabajando. Un subagente es un trabajador aparte con su propia ventana. Skill para criterio, subagente para trabajo que conviene aislar |
| E ¿Skill o MCP?                     | Distinto plano. MCP conecta herramientas y datos externos. La skill dice cómo y cuándo usarlos. Muchas veces conviven: la skill es el procedimiento y el MCP la puerta                |
| E ¿Y si mañana cambia el mecanismo? | El contenido no se pierde: lo que escribieron es criterio en texto. El formato del archivo puede cambiar; el criterio de cómo revisan un PR, no                                       |

## Preguntas incómodas

Las que salen en audiencias de desarrolladores y donde la respuesta defensiva se nota. Todas son del expositor, salvo la primera.

### "Lo probé y no me funcionó" · D

¿Qué decía la descripción? Te la leo de vuelta y te digo en treinta segundos si es eso.

Si la pega en el chat, **léela en voz alta y diagnósticala en vivo**. Es el mejor contenido posible del webinar y no se puede planear.

Si no la pega:

Si la traes a mano, pégala en el chat. Y si no, la respuesta más probable es la descripción, en ese orden: tercera persona, el cuándo, y las palabras que tecleas de verdad.

### "Esto es solo prompting con pasos extra" · E

En parte sí, y ahí está el punto. La diferencia es **quién** lo escribe y **cuántas veces**. Un prompt lo escribe cada persona cada vez. Una skill la escribe alguien una vez y la hereda todo el equipo. El mecanismo no es novedoso. Lo que cambia es que el criterio deja de depender de que alguien se acuerde de escribirlo.

### "¿No es esto amarrarse a un proveedor?" · E

El archivo es markdown. Si mañana se van a otra herramienta, lo que escribieron —cómo revisan un PR, qué exigen en un README— sigue siendo válido y portable. El formato del frontmatter es lo único específico, y son tres líneas.

## "¿Y la seguridad? Estoy metiendo instrucciones que se ejecutan solas" · E

Esta merece respuesta seria, no una evasión:

Es una preocupación correcta y hay tres cosas concretas. Una: una skill de un repositorio ajeno se **lee antes** de usarla, en particular su `allowed-tools` y sus scripts. Es código que alguien más escribió. Dos: nunca va un secreto dentro de una skill. Se versiona con el repositorio, y eso significa que va a terminar en el historial de git de todo el mundo. Tres: `allowed-tools` otorga permiso, no lo restringe. Si alguien esperaba usarlo como sandbox, no es lo que hace.

## "¿Cuánto contexto me come esto realmente?" · D

Córranle `/context` con sus skills instaladas y lo ven medido, que es mejor que cualquier número que yo les diga. El orden de magnitud: una descripción son dos o tres líneas y el cuerpo no existe hasta que se invoca. Lo que sí sorprende a la gente es que el cuerpo **se queda** después de invocarla, y por eso insistimos en cuerpos cortos.

## "En mi empresa no nos dejan instalar nada" · E

No hay nada que instalar. Es un archivo de texto en una carpeta. Si ya tienen Claude Code aprobado, las skills no agregan superficie nueva. Y si hay política de por medio, el nivel empresarial existe justamente para eso: skills administradas centralmente, que ganan sobre las personales.

## Si nadie pregunta nada

**No lo señales.** El silencio en el chat es normal en los primeros cuarenta minutos de un webinar y comentarlo lo empeora.

Toma una de estas cuatro y plantéala como si te la hubieran hecho:

1. "Me preguntan en el chat en qué se diferencia de un `CLAUDE.md`..." — la mejor para abrir, porque casi todos la tienen en la cabeza.
2. "Alguien pregunta cuántas skills debería escribir para empezar..." — permite meter el compromiso de una sola.
3. "Me llega la pregunta de por qué no se dispara la mía..." — enlaza directo con la demo 3.
4. "Preguntan si esto se puede compartir con el equipo..." — enlaza con el nivel de proyecto.

**No inventes una pregunta hostil que nadie hizo.** Suena a que te estás defendiendo de algo, y la audiencia lo nota.

## Lo que se anota para el correo

Todo lo que ninguno de los dos supo contestar va a una lista, y esa lista se responde en el correo de seguimiento del día siguiente. Quien la lleva es el demostrador, porque el expositor está vigilando el reloj.

No lo sé de memoria y prefiero no inventarlo. Lo anoto y va en el correo de seguimiento con la respuesta verificada.

Esa frase se puede decir dos veces en noventa minutos sin costo. A la tercera empieza a sonar a que no prepararon el tema.